Trabalho Prático 1: Confusão e Difusão em Imagens

Programação de Baixo Nível · Profs. Edson Moreno e Marcelo Cohen · 2026/2

Integrantes:	Guilherme de Lima Rosmann e Matheus Lopes Azambuja
Repositório GitHub:	https://github.com/DrakRosmann/bagunceitor
Vídeo (não listado):	[https://youtu.be/XXXXXXXXXXX]

1. Introdução

Este trabalho implementa, em C, um programa (`bagunceitor`) que "bagunça" uma imagem RGB aplicando duas transformações em memória: **confusão** (substituição dos valores dos bytes) e **difusão** (permutação das posições). Cada uma tem a sua operação inversa, e a imagem original é recuperada aplicando as inversas em ordem contrária:

$$\texttt{Original} \rightarrow C \rightarrow D \rightarrow \texttt{Bagunçada} \quad | \quad \texttt{Bagunçada} \rightarrow D^{-1} \rightarrow C^{-1} \rightarrow \texttt{Original}$$

A imagem é carregada pela biblioteca `stb_image` como um bloco contínuo de `largura × altura × 3` bytes (um `Pixel` = 3 bytes R, G, B). Todas as transformações trabalham diretamente nesse bloco, por meio de ponteiros.

2. A chave e o gerador pseudoaleatório

As duas transformações dependem de uma **chave** textual (padrão: `"progbn2026"`, ou informada na linha de comando). A chave passa pelo hash FNV-1a de 32 bits, misturado com uma constante diferente ("sal") para cada etapa, e o resultado é a semente de um gerador **xorshift32**:

```
x ^= x << 13;  x ^= x >> 17;  x ^= x << 5;
```

O gerador é determinístico: a mesma semente produz sempre a mesma sequência. É isso que permite às inversas reproduzirem exatamente os mesmos sorteios feitos na ida. Usar sais diferentes evita que a confusão e a difusão usem a mesma sequência de números.

3. Estratégia de confusão (C)

A imagem é tratada como um vetor de bytes, percorrido com um ponteiro `unsigned char *p` do início até `fim = p + w*h*3`. Para cada byte *x*, sorteia-se um número de 32 bits, de onde saem a chave do byte *k* (bits 0–7) e uma quantidade de rotação *r* (bits 8–10, de 0 a 7). O byte cifrado é obtido em quatro passos:

Passo	Ida (C)	Volta (C^{-1}), ordem contrária
1. XOR	`y1 = x ^ k`	`x = y1 ^ k`
2. Rotação	`y2 = rotl8(y1, r)`	`y1 = rotr8(y2, r)`
3. Multiplicação	`y3 = y2 · 157 mod 256`	`y2 = y3 · 181 mod 256`
4. Encadeamento	`c = y3 + c`$_{anterior}$ `mod 256`	`y3 = c − c`$_{anterior}$ `mod 256`

Justificativa da reversibilidade. Cada passo é uma bijeção no conjunto 0..255:

- o XOR com *k* é a sua própria inversa (`(x ^ k) ^ k = x`);
- a rotação circular de 8 bits não perde bits, e rodar *r* bits para a direita desfaz a rotação para a esquerda;
- como 157 é ímpar, ele tem inverso multiplicativo módulo 256: 157 · 181 = 28417 = 111 · 256 + 1. O programa calcula esse inverso em tempo de execução, com o método de Newton (`inverso_mod256`);
- a soma módulo 256 é desfeita pela subtração. Como $c_{anterior}$ é o byte **já cifrado**, ele está disponível na imagem bagunçada, e a volta consegue usá-lo. O primeiro byte usa como "anterior" um valor derivado da chave.

Por que essa combinação. Só o XOR já esconde as cores, mas regiões uniformes (como o céu) ainda apareceriam quando a chave se repetisse. A multiplicação e a rotação misturam os bits dentro do byte, e o encadeamento faz cada byte cifrado depender de todos os anteriores, de modo que dois pixels iguais quase nunca geram o mesmo valor. Na figura 1 (b), aplicando apenas C, a imagem já vira ruído, mesmo sem que nenhum pixel tenha mudado de lugar.

```
while (p < fim) {
  uint32_t sorteio = xorshift32(&estado);
  unsigned char k = sorteio & 0xFF;
  int r = (sorteio >> 8) & 7;
  unsigned char y = *p ^ k;
  y = rotl8(y, r);
  y = (unsigned char)(y * MULT);
  y = (unsigned char)(y + anterior);
  *p++ = y;
  anterior = y;
}
```

4. Estratégia de difusão (D)

A difusão só muda posições, nunca valores, e atua em duas granularidades:

4.1 Pixel inteiro: permutação por Fisher–Yates com chave

Gera-se um vetor `perm` com os índices 0..n−1 (n = w·h) e ele é embaralhado pelo algoritmo de Fisher–Yates, usando o xorshift32 para sortear as trocas. O resultado é uma permutação, isto é, uma correspondência um-para-um entre as posições de origem e de destino. A ida faz `destino[i] = origem[perm[i]]`. A volta gera o **mesmo** vetor (mesma chave) e faz o mapeamento reverso, `destino[perm[i]] = origem[i]`, devolvendo cada pixel ao lugar de onde saiu.

```
/* D */    for (size_t* p = perm; p < perm + n; p++) *d++ = *(src->pixels + *p);
/* D^-1 */ for (size_t* p = perm; p < perm + n; p++) *(dst->pixels + *p) = *s++;
```

4.2 Byte: rotação dos canais dentro do pixel

Depois da permutação, os 3 bytes de cada pixel são rodados 0, 1 ou 2 posições (RGB → GBR → BRG), conforme o sorteio. A volta roda os canais `(3 − passo) mod 3` posições, o que desfaz a rotação. Esse

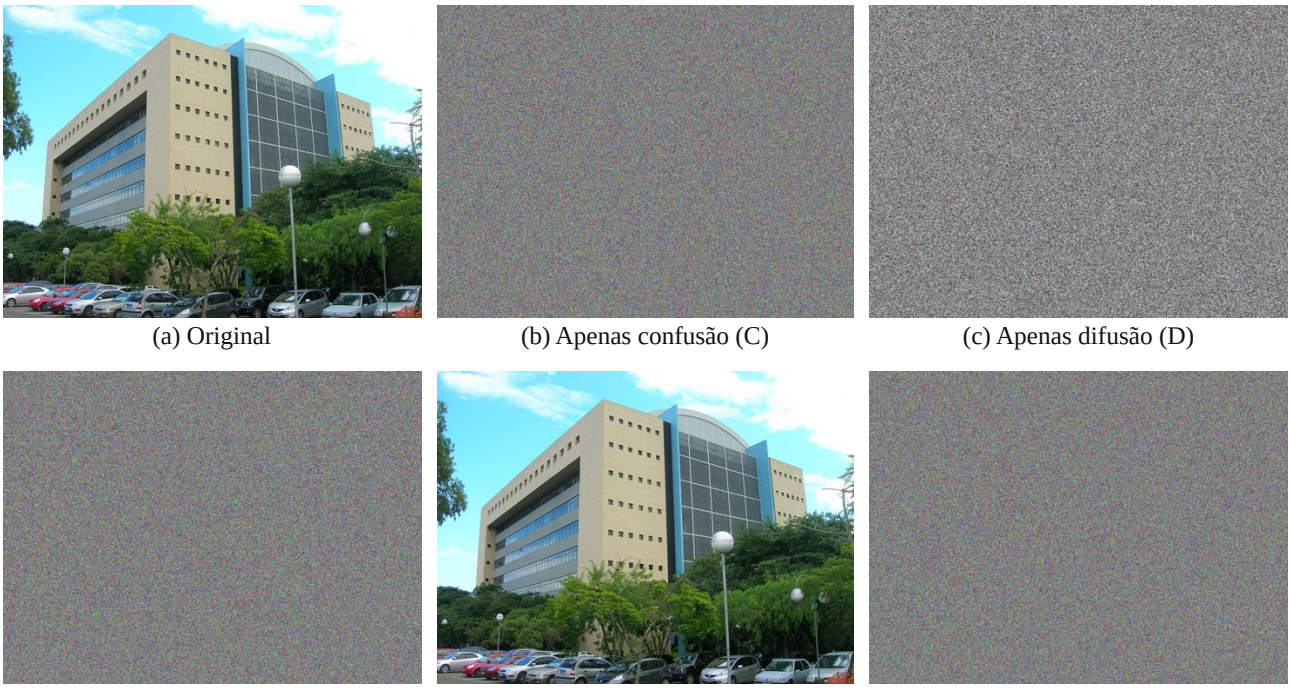
passo é feito antes da permutação inversa, já que na ida ele foi o último.

Justificativa. A permutação de pixels destrói toda a estrutura espacial (contornos, formas). Como os valores não mudam, o histograma de cada canal se mantém. A rotação de canais mistura também os histogramas entre R, G e B. Na figura 1 (c), aplicando apenas D, as cores médias ainda aparecem (predomínio de tons acinzentados do prédio e do céu), mas nenhum contorno sobrevive.

5. Manipulação de memória

- A imagem é um único bloco alocado dinamicamente; a função `nova_imagem` usa `malloc` + `memset` e as outras liberam com `free` tudo que alocaram (a execução no `valgrind --leak-check=full` não reportou erros nem vazamentos).
- A confusão converte `Pixel*` para `unsigned char*` e percorre os bytes com aritmética de ponteiros (`*p++`, comparação `p < fim`).
- A difusão usa `Pixel*` para copiar os 3 bytes de uma vez e acessa índices com `*(base + deslocamento)`. A rotação de canais enxerga o mesmo pixel ao mesmo tempo como `Pixel*` e como `unsigned char*`.
- As contagens usam `size_t`, o que evita estouro de `int` em imagens grandes.
- A imagem é carregada forçando 3 canais (`stbi_load(..., 3)`), o que garante que o layout da memória coincide com a `struct Pixel` mesmo para PNGs com transparência ou em tons de cinza.

6. Resultados e prova de recuperação



(a) Original (b) Apenas confusão (C) (c) Apenas difusão (D)

(d) Bagunçada (C e D) (e) Recuperada (D^{-1} e C^{-1}) (f) Tentativa com chave errada

Figura 1: etapas sobre `predio32.jpg` (1280 × 960), chave "progbn2026".

A semelhança visual não basta como prova. Por isso o programa compara a original e a recuperada **byte a byte** (função `compara`) e também com `memcmp`. Trecho da saída:

```
Original (0x7f14c09e2010):
[A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A7 E5 FA] [A7 E5
FA]
Baguncada (0x7f14c065d010):
[6D 0A F3] [48 6F DF] [85 B2 FB] [0A 39 C2] [DC 39 77] [AC 89 B7] [69 6E 33] [59 17
F7]
Recuperada (0x7f14c02d8010):
[A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A6 E4 FB] [A7 E5 FA] [A7 E5
FA]

Bytes diferentes (original x baguncada) : 3672003 de 3686400 (99.61%)
Bytes diferentes (original x recuperada): 0 de 3686400
memcmp(original, recuperada) = 0 -> RECUPERACAO EXATA
Com chave errada: 3671935 bytes diferentes
```

Os seis primeiros pixels da original são idênticos (céu), mas na bagunçada todos são diferentes, o que mostra o efeito do encadeamento. Os 0,39% de bytes "iguais" entre original e bagunçada correspondem ao acaso esperado (1/256 ≈ 0,39%). A recuperação também funciona a partir do arquivo: como o PNG não tem perdas, `./bagunceitor -d baguncada.png progbn2026` gera um arquivo idêntico a `recuperada.png`. Por outro lado, a bagunçada **não** pode ser salva em JPEG, porque a compressão com perdas alteraria os bytes.

7. Como compilar e executar

```
make                                        # compila
./bagunceitor predio32.jpg [chave]          # demonstração completa (gera os PNGs e
confere)
./bagunceitor -e imagem.jpg [chave] [saida]  # só bagunça
./bagunceitor -d baguncada.png [chave] [saida]  # só recupera
make clean                                  # apaga .o e executável
```

8. Uso de Inteligência Artificial

[Descrever aqui, com honestidade, como a IA foi usada. Exemplo: "Utilizamos o assistente Claude (Anthropic) para gerar uma primeira versão do código e do relatório. Revisamos, testamos e entendemos todo o conteúdo, e conseguimos explicar cada etapa no vídeo." Ajustar conforme o que o grupo realmente fez.]

9. Conclusão

As duas estratégias autorais (confusão encadeada XOR–rotação–multiplicação e difusão por permutação de pixels com rotação de canais) bagunçaram a imagem até virar ruído visual. Como cada operação tem uma inversa exata e o programa aplica as inversas na ordem contrária, a imagem original é recuperada integralmente, sem diferença em nenhum dos 3.686.400 bytes.